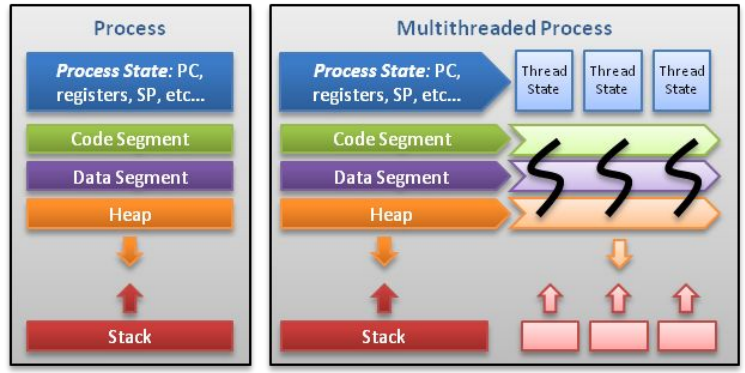


Inter-Process Communication

Advanced Embedded Programming

Process

- A process is an instance of a program that is being executed by one or more threads.
  - A process has its own memory space that the process is allowed to access.
    - Processes don't share their memory spaces.
    - But there are different mechanisms to exchange data between processes
- Every process has an ID and you can get the list of running processes using commands like **ps**, **top** or programs like **htop** in a terminal.
- To terminate a process you can use the **kill**, **pkill** or **killall** commands.
- A process can create and manage child processes using the **fork** and **wait/waitpid** functions which are system calls in Unix-like operating systems



Process - fork()

- When **fork**() is called, it essentially duplicates the calling process, resulting in two identical processes, known as the parent process and the child process.
- The child process is an exact copy of the parent process, with its own memory space, file descriptors, and program counter.
- In the parent process, fork() returns the process ID (PID) of the newly created child process and in the child process, it returns 0.
  - This differentiation in return values allows the parent and child processes to execute different code paths based on the return value.
  - On failure, -1 is returned in the parent, no child process is created, and errno is set to indicate the error.

```
#include <unistd.h>

pid_t fork(void);
```

Process - system calls

- Both the child and the parent processes continue from the same place in the code where the fork() call was used.
- **fork**() is used in multiprocessing and as the starting point for creating parallel processes.
- **getpid**() returns the process ID(PID) and **getppid**() returns the process ID of the parent of the calling process.

```
#include <unistd.h>

pid_t getpid(void);
pid_t getppid(void);
```

- wait() and waitpid() are used to wait for state changes in a child of the calling process, and obtain information about the child whose state has changed
  - **wait**() suspends execution of the calling thread until one of its children terminates.
  - **waitpid**() suspends execution of the calling thread until a specific child has changed state. By default, it waits only for terminated children, but this behavior is modifiable.

Process

- **wait**() on success returns the process ID of the terminated child; on failure, returns -1
- **waitpid**(): on success, returns the process ID of the child whose state has changed.
- if **WNOHANG** was specified as an option and one or more child(ren) specified by pid exist, but have not yet changed state, then 0 is returned. On failure, -1 is returned.
- If a parent process terminates before its child processes, the child processes become **orphaned** and **systemd** (init process) adopts it to ensure that resources associated with orphaned child processes are properly released.
- If the parent process does not call **wait**() or **waitpid**() to collect the exit status of its child processes, they can become **zombie** processes. A zombie process is a child process that has terminated but still has an entry in the process table to allow the parent to retrieve its exit status.

Process - fork()

```c
// An example of an orphaned process
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main(void) {
  printf("Main PID: %d\n", getpid());
  printf("Main PPID: %d\n", getppid());
  pid_t child_pid = fork();
  if (child_pid == -1) { perror("Fork failed"); }
  else if (child_pid == 0) { /* The child process */
    printf("Child PID: %d\n", getpid());
    printf("Child PPID: %d\n", getppid());
    sleep(5); // Simulate some work in the child process
    printf("Child process is done.\n");
  } else { /* The parent process */
    printf("Child PID: %d\n", child_pid);
    printf("Parent PID: %d\n", getppid());
    printf("\nThe parent process is running and will terminate soon.\n");
    // Parent process terminates without waiting for the child.
  }
  return 0;
}
```

```c
// A zombie process example. it can observed using: ps -e | grep defunct in a separate terminal
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>

int main(void) {
  printf("Parent process (PID %d) is starting.\n", getpid());
  pid_t child_pid = fork();
  if (child_pid == -1) { perror("Fork failed"); }
  else if (child_pid == 0) { /* The child process */
    printf("Child process (PID %d) is running.\n", getpid());
    sleep(1); // Simulate some work
    printf("Child process (PID %d) is done.\n", getpid());
  } else { /* The parent process */
    int status;
    printf("Parent process is waiting for the child process (PID %d) to finish.\n", child_pid);
    sleep(10); // Parent sleeps for a while. The child process may become a zombie at this point.
    printf("Parent process (PID %d) is done.\n", getpid());
    wait(&status); // Retrieve the child's exit status
}

  return 0;
}
```

Process - Multiprocessing

```c
// An example of multiprocessing
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>

#define PROCESSES_NUM 4

void perform_work(int process_index) {
  printf("Child process %d (PID %d) is doing some work.\n", process_index, getpid());
  sleep(2); // Simulate some work
  printf("Child process %d (PID %d) is done.\n", process_index, getpid());
}

int main(void) {
printf("Parent process (PID %d) is starting.\n", getpid());
for (int i = 1; i <= PROCESSES_NUM; i++) {
    pid_t child_pid = fork();
    if (child_pid == -1) { perror("Fork failed"); }
    else if (child_pid == 0) { /* The child process */
    perform_work(i);
    return 0; // or exit(0); The child process exits after work is done
  }
}
```

```c
// The parent process
for (int i = 0; i < PROCESSES_NUM; i++) {
  int status;
  pid_t pid = wait(&status); // Wait for child processes to finish
  printf("PID: %d, Exit Code: %d\n", pid, WEXITSTATUS(status));
}

printf("Parent process (PID %d) is done.\n", getpid());

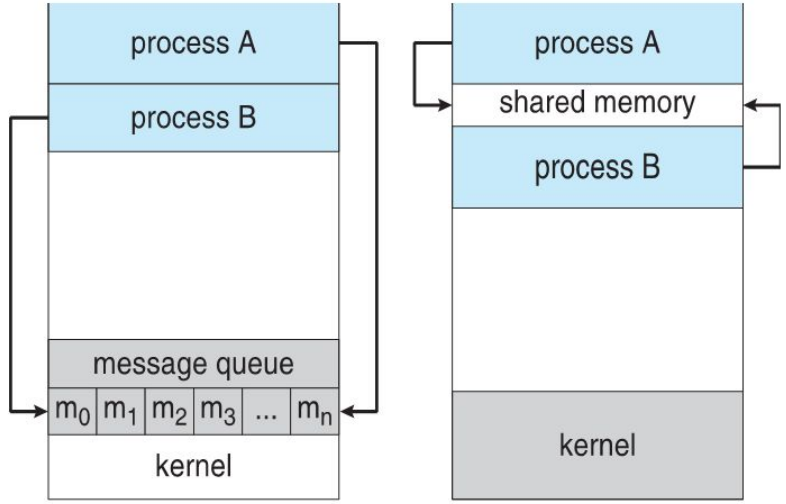
return 0;
}
```

Process - Communication

- Processes don't share their memory spaces. But there are different mechanisms that allow different processes to communicate and share data with each other.
- **Pipes** are communication channels that allow exchanging data between processes. There are two types of Pipe: unnamed pipes (unidirectional) and named pipes (bidirectional). Named pipes are FIFO files.
- **Sockets** are used for communication over a network or between processes on the same machine(local sockets or UNIX-domain sockets). They provide bidirectional communication.
  - E.g. The TCP/IP socket communication in the dashboard project.
- **File Locking**: Processes can use file locks to coordinate access to shared files and prevent data corruption.

Process - Inter-Process Communication

- **POSIX IPC** mechanisms are the Posix standard implementation of the **System V IPC** mechanisms. **System V IPC** is a set of inter-process communication mechanisms provided by Unix-like operating systems.
- The **POSIX IPC** mechanisms are:
  - **Message Queues**: allow processes to exchange messages through named queues. Messages in a queue have priorities associated with them.



  - **Shared Memory**: allows multiple processes to share a common memory segment. This is the most efficient IPC methods since it involves direct memory access.
  - **Semaphores**: are a synchronization mechanism that can be used for IPC.

IPC - Unnamed Pipe Example

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>

/* Write COUNT copies of MESSAGE to STREAM */
static void writer(const char *message, int count, FILE *stream) {
  for (; count > 0; --count) {
    /* Write the message to the stream, and send it off immediately. */
    fprintf(stream, "%s\n", message);
    fflush(stream);
    sleep(1); /* Sleep a while. */
  }
}

/* Read random strings from the stream as long as possible. */
static void reader(FILE *stream) { char buffer[1024];
  /* Read until we hit the end of the stream. */
  while (!feof(stream) && !ferror(stream) &&
         (NULL != fgets(buffer, sizeof(buffer), stream))) {
    fputs(buffer, stdout);
  }
}
```

```c
int main(void) {   int fds[2];
  /* Create a pipe. File descriptors for the two ends of the pipe are placed in fds. */
  if (-1 == pipe(fds)) { perror("Pipe creation failed"); exit(1); }
  pid_t pid = fork(); /* Fork a child process. */
  if (pid == -1) { perror("Fork failed"); exit(EXIT_FAILURE); }
  else if (pid == 0) { /* The child process. */
    close(fds[1]); /* Close our copy of the write end of the file descriptor. */
    /* Convert the read file descriptor to a FILE object, and read from it. */
    FILE *stream = fdopen(fds[0], "r");
    reader(stream);
    close(fds[0]);
  } else { /* The parent process. */
    close(fds[0]); /* Close our copy of the read end of the file descriptor. */
    /* Convert the write file descriptor to a FILE object, and write to it. */
    FILE *stream = fdopen(fds[1], "w");
    writer("Hello, world.", 5, stream);
     close(fds[1]);
    int status; pid = wait(&status);
    printf("PID: %d, Exit Code: %d\n", pid, WEXITSTATUS(status));
  }
  return 0;
}
```

IPC - Named Pipe(FIFO) Example

```cpp
// reader.cpp: A named pipe(FIFO file) example.
#include <ctime>
#include <fcntl.h>
#include <cstring>
#include <unistd.h>
#include <iostream>
#include <sys/stat.h>

constexpr int MAX_LEN{32};
const char *const FIFO_NAME{"/tmp/myfifo"}; // FIFO file path

int main(void) { srand(time(nullptr));
  (void)mkfifo(FIFO_NAME, 0666); // Create the FIFO if it doesn't exist
  int fd; char buffer[MAX_LEN + 1] = {0};

  while (1) {
    fd = open(FIFO_NAME, O_RDONLY); // First open in read only and read
    if (fd == -1) { std::cout << "Failed to open the FIFO..." << std::endl; break; }
    if (sizeof(buffer) != read(fd, buffer, sizeof(buffer)))
    { std::cout << "Failed to read from the FIFO..." << std::endl; close(fd); break; }
    close(fd); std::cout << "Received:" << buffer << std::endl;
    for (int i = 0; i < sizeof(buffer) - 1; i++) { buffer[i] = 'a' + (rand() % 26); }
    fd = open(FIFO_NAME, O_WRONLY); // Open in write mode and write
    if (fd == -1) { std::cout << "Failed to open the FIFO..." << std::endl; break; }
    if (sizeof(buffer) != write(fd, buffer, sizeof(buffer)))
    { std::cout << "Failed to write to the FIFO..." << std::endl; close(fd); break; }
    close(fd);
  }

  return 0;
}
```

```cpp
// writer.cpp: A named pipe(FIFO file) example.
#include <ctime>
#include <fcntl.h>
#include <cstring>
#include <unistd.h>
#include <iostream>
#include <sys/stat.h>

constexpr int MAX_LEN{32};
const char *const FIFO_NAME{"/tmp/myfifo"}; // FIFO file path

int main(void) { std::srand(time(nullptr));
  (void)mkfifo(FIFO_NAME, 0666); // Create the FIFO if it doesn't exist
  int fd; char buffer[MAX_LEN + 1] = {0};

  while (1) {
    fd = open(FIFO_NAME, O_WRONLY); // Open FIFO for write only
    if (fd == -1) { std::cout << "Failed to open the FIFO..." << std::endl; break; }
    for (int i = 0; i < sizeof(buffer) - 1; i++) { buffer[i] = 'A' + (rand() % 26); }
    // Write the buffer on FIFO and close it
    if (sizeof(buffer) != write(fd, buffer, sizeof(buffer)))
    { std::cout << "Failed to write to the FIFO..." << std::endl; close(fd); break; }
    close(fd); fd = open(FIFO_NAME, O_RDONLY); // Open FIFO for Read only
    if (fd == -1) { std::cout << "Failed to open the FIFO..." << std::endl; break; }
    if (sizeof(buffer) != read(fd, buffer, sizeof(buffer)))
    { std::cout << "Failed to read from the FIFO..." << std::endl; close(fd); break; }
    close(fd); std::cout << "Received:" << buffer << std::endl; sleep(1);
  }

  return 0;
}
```

IPC - Local or UNIX-domain Socket Example

```cpp
// server.cpp
#include <cstring>
#include <unistd.h>
#include <sys/un.h>
#include <iostream>
#include <sys/types.h>
#include <sys/socket.h>

constexpr int MAX_LEN{32}; const char *const SOCKET_NAME{"/tmp/local_socket"};

int main(void) { unlink(SOCKET_NAME); // Remove the socket file if it exists
  int server_socket = socket(AF_LOCAL, SOCK_STREAM, 0); // Create a socket
  if (server_socket == -1) { std::cout << "Failed to create the socket..." << std::endl; std::exit(EXIT_FAILURE); }
  sockaddr_un servaddr{0}; // Set up the address structure
  servaddr.sun_family = AF_LOCAL;
  strncpy(servaddr.sun_path, SOCKET_NAME, sizeof(servaddr.sun_path) - 1);
  // Bind the socket to the specified address
  if (0 != bind(server_socket, (sockaddr *)&servaddr, sizeof(servaddr))) { std::cout << "Failed to bind to the socket..." << std::endl; close(server_socket); std::exit(EXIT_FAILURE); }
  if (0 != listen(server_socket, 1)) { /* only one connection */ std::cout << "Failed to listen to the port..." << std::endl; close(server_socket); std::exit(EXIT_FAILURE); }
  std::cout << "Server is waiting for connections..." << std::endl;
  // Accept a connection
  int client_socket = accept(server_socket, nullptr, nullptr);
  if (client_socket == -1) { std::cout << "Failed to accept the connection..." << std::endl; close(server_socket); exit(EXIT_FAILURE); }
  else { std::cout << "Server acccepted the client..." << std::endl; }
  while (1) { char buffer[MAX_LEN + 1]{0};
    // Read the message from client and store it in buffer
    if (1 > read(client_socket, buffer, MAX_LEN)) { std::cout << "Failed to read ..." << std::endl; close(client_socket); break; }
    size_t len = strlen(buffer); for (size_t i = 0; i < len; i++) { buffer[i] = toupper(buffer[i]); }
    // Send buffer to client
    if (len != write(client_socket, buffer, len)) { std::cout << "Failed to write ..." << std::endl; close(client_socket); break; }
  }
  close(server_socket); unlink(SOCKET_NAME); // Remove the socket file
  return 0;
}
```

IPC - Local or UNIX-domain Socket Example

```cpp
// client.cpp
#include <ctime>
#include <cstring>
#include <unistd.h>
#include <sys/un.h>
#include <iostream>
#include <sys/types.h>
#include <sys/socket.h>

constexpr int MAX_LEN{32}; const char *const SOCKET_NAME{"/tmp/local_socket"};

int main(void) { std::srand(std::time(nullptr)); int client_socket = socket(AF_LOCAL, SOCK_STREAM, 0); // Create a socket
  if (client_socket == -1) { std::cout << "Failed to create the socket..." << std::endl; std::exit(EXIT_FAILURE); }
  sockaddr_un servaddr{0}; // Set up the address structure
  servaddr.sun_family = AF_LOCAL;
  strncpy(servaddr.sun_path, SOCKET_NAME, sizeof(servaddr.sun_path) - 1);
  if (0 != connect(client_socket, (sockaddr *)&servaddr, sizeof(servaddr))) {
    std::cout << "Connection to the server failed..." << std::endl; close(client_socket); std::exit(EXIT_FAILURE);
  }
  while (1) { char buffer[MAX_LEN + 1]{0}; int len = 1 + std::rand() % MAX_LEN;
    for (int i = 0; i < len; i++) { buffer[i] = 'a' + std::rand() % 26; }
    std::cout << "Sent :" << buffer << std::endl;
    if (len != write(client_socket, buffer, len)) { std::cout << "Failed to write ..." << std::endl; break; }
    bzero(buffer, sizeof(buffer));
    if (len != read(client_socket, buffer, len)) { std::cout << "Failed to read ..." << std::endl; break; }
    std::cout << "Received:" << buffer << std::endl << std::endl; sleep(1);
  }
  close(client_socket); // Close the socket

  return 0;
}
```

IPC - File Locking Example

```cpp
// reader.cpp: An example of File Locking mechanism
#include <fcntl.h>
#include <unistd.h>
#include <iostream>

int main(void) {
  const char *file_path = "lockfile";
  // Open the file for reading
  int fd = open(file_path, O_RDONLY);
  if (fd == -1) { std::cout << "Failed to open the file." << std::endl; std::exit(1); }
  // Lock the file for reading
  flock lock;
  lock.l_type = F_RDLCK; // Read lock
  lock.l_whence = SEEK_SET;
  lock.l_start = 0;
  lock.l_len = 0;
  if (fcntl(fd, F_SETLKW, &lock) == -1)
  { std::cout << "Failed to lock the file." << std::endl; close(fd); std::exit(1); }
  char buffer[64] = {0};
  // Read data from the locked file
  if (0 < read(fd, buffer, sizeof(buffer) - 1))
  { std::cout << "Read from locked file: " << buffer << std::endl; }
  else { std::cout << "Failed to read from the file." << std::endl; }
  lock.l_type = F_UNLCK; // Unlock the file
  if (-1 == fcntl(fd, F_SETLK, &lock))
  { std::cout << "Failed to unlock the file." << std::endl; }
  close(fd); // Close the file

  return 0;
}
```

```cpp
// writer.cpp: An example of File Locking mechanism
#include <fcntl.h>
#include <cstring>
#include <unistd.h>
#include <iostream>

int main(void) {
  const char *file_path = "lockfile";
  const char *message = "This is a locked file example.";
  // Open the file for writing
  int fd = open(file_path, O_WRONLY | O_CREAT, S_IRUSR | S_IWUSR);
  if (fd == -1) { std::cout << "Failed to open the file." << std::endl; std::exit(1); }
  // Attempt to lock the file
  flock lock;
  lock.l_type = F_WRLCK; // Write lock
  lock.l_whence = SEEK_SET;
  lock.l_start = 0; lock.l_len = 0;
  if (-1 == fcntl(fd, F_SETLKW, &lock))
  { std::cout << "Failed to lock the file." << std::endl; close(fd); std::exit(1); }
  // Write data to the locked file
  if (-1 == write(fd, message, strlen(message)))
  { std::cout << "Failed to write to the file." << std::endl; close(fd); std::exit(1); }
  sleep(10); // Sleep 10s while the file is locked
  lock.l_type = F_UNLCK; // Unlock the file
  if (-1 == fcntl(fd, F_SETLKW, &lock))
  { std::cout << "Failed to unlock the file." << std::endl; }
  close(fd); // Close the file

  return 0;
}
```

IPC - POSIX Message Queue Example

```cpp
// sender.cpp: An example of POSIX IPC Message Queues.
// Look at: https://man7.org/linux/man-pages/man7/mq_overview.7.html
// To build: g++ sender.cpp -lrt -o sender, To show: cat /dev/mqueue/mymq
#include <cstring>
#include <mqueue.h>
#include <iostream>

constexpr int MAX_MSG_SIZE{1024};
const char *const QUEUE_NAME{"/mymq"};

int main(void) {
  mq_attr attr{0}; // Set up message queue attributes
  attr.mq_maxmsg = 10;
  attr.mq_msgsize = MAX_MSG_SIZE;
  // Open the message queue
  mqd_t mq = mq_open(QUEUE_NAME, O_WRONLY | O_CREAT, 0666, &attr);
  if (mq == -1)
  { std::cout << "Failed to open the queue" << std::endl; std::exit(1); }
  char buffer[MAX_MSG_SIZE];
  while (1) { std::memset(buffer, 0, sizeof(buffer));
    std::cout << "Enter a message: "; std::cin >> buffer;
    // Send the message
    if (-1 == mq_send(mq, buffer, strlen(buffer) + 1, 0))
    { std::cout << "Failed to send the message" << std::endl; break; }
    if (0 == std::strcmp(buffer, "exit")) { break; }
  }
  mq_close(mq); // Close the message queue
  mq_unlink(QUEUE_NAME); // Remove the message queue

  return 0;
}
```

```cpp
// receiver.cpp: An example of POSIX IPC Message Queues
// Look at: https://man7.org/linux/man-pages/man7/mq_overview.7.html
// To build: g++ receiver.cpp -lrt -o receiver, To show: cat /dev/mqueue/mymq
#include <cstring>
#include <mqueue.h>
#include <iostream>

constexpr int MAX_MSG_SIZE{1024};
const char *const QUEUE_NAME{"/mymq"};

int main(void) {
  // Open the message queue for reading
  mqd_t mq = mq_open(QUEUE_NAME, O_RDONLY);
  if (mq == -1) { std::cout << "Failed to open the queue" << std::endl; std::exit(1); }
  char buffer[MAX_MSG_SIZE];
  while (1) {
    std::memset(buffer, 0, sizeof(buffer));
    if (-1 == mq_receive(mq, buffer, MAX_MSG_SIZE, nullptr))
    { std::cout << "Failed to receive a message" << std::endl; break; }
    std::cout << "Received message: " << buffer << std::endl;
    if (0 == std::strcmp(buffer, "exit")) { break; }
}

// Close the message queue
mq_close(mq);

return 0;
}
```

IPC - POSIX Named Shared Memory Example

```cpp
// main.cpp: An example of POSIX IPC Shared Memory.
// Look at: https://man7.org/linux/man-pages/man7/shm_overview.7.html
// To build: g++ main.cpp -lrt -o main
// While the program is sleep run: cat /dev/shm/myshm
#include <fcntl.h>
#include <cstring>
#include <unistd.h>
#include <iostream>
#include <sys/stat.h>
#include <sys/mman.h>

constexpr int SHM_SIZE{1024};
const char *const SHM_NAME{"/myshm"};

int main(void) {
  // Create a shared memory object
  int shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
  if (shm_fd == -1) {
    std::cout << "Failed to create a shared memory object" << std::endl;
    std::exit(1);
  }

  // Configure the size of the shared memory segment
  if (-1 == ftruncate(shm_fd, SHM_SIZE)) {
    std::cout << "Failed to set size of shared memory object" << std::endl;
    close(shm_fd); shm_unlink(SHM_NAME); std::exit(1);
  }

  // Map the shared memory segment into the address space of this process
  char *shmp = (char *)mmap(nullptr, SHM_SIZE, PROT_READ | PROT_WRITE,
                            MAP_SHARED, shm_fd, 0);
  if (shmp == MAP_FAILED) {
    std::cout << "Failed to map the shared memory segment" << std::endl;
    close(shm_fd); shm_unlink(SHM_NAME); std::exit(1);
  }

  // Write a message to the shared memory
  std::strcpy(shmp, "Hello, Shared Memory!");
  std::cout << "Message written to shared memory: " << shmp << std::endl;

  pid_t pid = fork(); // Fork a child process

  if (pid == -1) { std::cout << "Failed to fork the child process" << std::endl; }
  else if (pid == 0) { /* Child process reads from shared memory */
    std::cout << "Child process reads from shared memory: "
              << shmp << std::endl;
  }

  // Sleep a while
  sleep(10);

  // Unmap and close the shared memory
  munmap(shmp, SHM_SIZE); close(shm_fd);

  // Remove the shared memory object
  (void)shm_unlink(SHM_NAME);

  return 0;
}
```

IPC - POSIX Semaphore (In An Unnamed Shared Memory)

```cpp
// main.cpp: An example of a POSIX IPC Unnamed Semaphore.
// Look at: https://man7.org/linux/man-pages/man7/sem_overview.7.html
// To build: g++ main.cpp -lpthread -lrt -o main
#include <fcntl.h>
#include <unistd.h>
#include <iostream>
#include <sys/stat.h>
#include <sys/wait.h>
#include <sys/mman.h>
#include <semaphore.h>

constexpr int PROCESSES_NUM{5};

int main(void) {
  // The semaphore object shall be located in a region of shared memory
  sem_t *semaphore = (sem_t *)mmap(nullptr, sizeof(sem_t), PROT_READ |
                     PROT_WRITE, MAP_ANONYMOUS | MAP_SHARED, 0, 0);
  if (semaphore == MAP_FAILED) {
    std::cout << "Failed to create the semaphore." << std::endl; std::exit(1);
  }
  if (-1 == sem_init(semaphore, 1, 1)) { (void)sem_destroy(semaphore);
    std::cout << "Failed to initialize the semaphore" << std::endl; std::exit(1);
  }
  // Fork multiple child processes
  for (int i = 0; i < PROCESSES_NUM; i++) { pid_t pid = fork();
    if (pid == -1) {
      std::cout << "Failed to fork a child process." << std::endl; std::exit(1);
    }
  else if (pid == 0) { /* Child process */
      (void)sem_wait(semaphore);
      std::cout << "Child " << i << " has acquired the semaphore!"
                << std::endl << std::flush;
      sleep(2); // Simulate a work
      (void)sem_post(semaphore);
      std::exit(0);
    }
  }
  // Wait for child processes to finish
  for (int i = 0; i < PROCESSES_NUM; i++) { (void)wait(nullptr); }
  (void)sem_destroy(semaphore); // Destroy the semaphore
  // Unmap the shared memory
  (void)munmap(semaphore, sizeof(sem_t));

  return 0;
}
```

IPC - POSIX Named Semaphore Example

```
// ping.cpp: An example of a POSIX IPC Named Semaphore.
// To build: g++ ping.cpp -lpthread -lrt -o ping.
// Named semaphores are mounted under /dev/shm: ls /dev/shm
#include <fcntl.h>
#include <unistd.h>
#include <iostream>
#include <sys/stat.h>
#include <sys/wait.h>
#include <sys/mman.h>
#include <semaphore.h>

int main(void) { constexpr int ITERATIONS{10};
  const char *const SEM_PING{"/ping"}; (void)sem_unlink(SEM_PING);
  const char *const SEM_PONG{"/pong"}; (void)sem_unlink(SEM_PONG);

  sem_t *sem_ping = sem_open(SEM_PING, O_CREAT | O_RDWR, 0666, 1);
  sem_t *sem_pong = sem_open(SEM_PONG, O_CREAT | O_RDWR, 0666, 0);
  if ((sem_ping == SEM_FAILED) || (sem_pong == SEM_FAILED))
  { std::cout << "Failed to create the semaphores." << std::endl; }
  else { for (int i = 0; i < ITERATIONS; i++) {
       (void)sem_wait(sem_ping);
       std::cout << "Ping" << std::endl; sleep(1);
       (void)sem_post(sem_pong);
     }

     (void)sem_destroy(sem_ping); (void)sem_destroy(sem_pong);
     (void)sem_unlink(SEM_PING); (void)sem_unlink(SEM_PONG);
  }

  return 0;
}
```

```
// pong.cpp: An example of a POSIX IPC Named Semaphore.
// To build: g++ pong.cpp -lpthread -lrt -o pong.
// Named semaphores are mounted under /dev/shm: ls /dev/shm
#include <fcntl.h>
#include <unistd.h>
#include <iostream>
#include <sys/stat.h>
#include <sys/wait.h>
#include <sys/mman.h>
#include <semaphore.h>

int main(void) { constexpr int ITERATIONS{10};
  const char *const SEM_PING{"/ping"};
  const char *const SEM_PONG{"/pong"};
  // Open the shared semaphores
  sem_t *sem_ping = sem_open(SEM_PING, O_RDWR, 0666, 1);
  sem_t *sem_pong = sem_open(SEM_PONG, O_RDWR, 0666, 0);
  if ((sem_ping == SEM_FAILED) || (sem_pong == SEM_FAILED))
  { std::cout << "Failed to create the semaphores." << std::endl; }
    else { for (int i = 0; i < ITERATIONS; i++) {
      (void)sem_wait(sem_pong);
      std::cout << "Pong" << std::endl;
      (void)sem_post(sem_ping);
    }

    (void)sem_destroy(sem_ping);
    (void)sem_destroy(sem_pong);
  }

  return 0;
}
```

IPC - POSIX Mutex, Condition Variable and Shared Memory

```
// ping.cpp: To build: g++ ping.cpp -lpthread -lrt -o ping. Run ping before pong.
#include <fcntl.h>
#include <unistd.h>
#include <iostream>
#include <pthread.h>
#include <sys/mman.h>
#include <semaphore.h>

struct ping_pong_t { int value; pthread_cond_t cv; pthread_mutex_t mtx; };

int main(void) { constexpr int ITERATIONS{10}; constexpr int PING{0}, PONG{1};
const char *const SHM_NAME{"/ping_pong"}; shm_unlink(SHM_NAME);
int shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
if (shm_fd == -1)  { std::cout << "Failed to create a shared memory" << std::endl; std::exit(1); }
if (-1 == ftruncate(shm_fd, sizeof(ping_pong_t))) { std::cout << "Failed to set size of shared
memory object" << std::endl; close(shm_fd); shm_unlink(SHM_NAME); std::exit(1); }
ping_pong_t *ping_pong = (ping_pong_t *)mmap(nullptr, sizeof(ping_pong_t), PROT_READ |
PROT_WRITE, MAP_SHARED, shm_fd, 0);
if (ping_pong == MAP_FAILED) { std::cout << "Failed to map the shared memory segment" <<
std::endl; close(shm_fd); shm_unlink(SHM_NAME); std::exit(1); }
ping_pong->value = PING; pthread_mutexattr_t mtxattr; pthread_mutexattr_init(&mtxattr);
pthread_mutexattr_setpshared(&mtxattr, PTHREAD_PROCESS_SHARED);
pthread_mutex_init(&ping_pong->mtx, &mtxattr);
pthread_condattr_t cvattr; pthread_condattr_init(&cvattr); pthread_condattr_setpshared(&cvattr,
PTHREAD_PROCESS_SHARED); pthread_cond_init(&ping_pong->cv, &cvattr);
for (int i = 0; i < ITERATIONS; i++) { pthread_mutex_lock(&ping_pong->mtx);
while (ping_pong->value != PING) { pthread_cond_wait(&ping_pong->cv, &ping_pong->mtx); }
std::cout << "Ping" << std::endl; ping_pong->value = PONG;
pthread_cond_signal(&ping_pong->cv); pthread_mutex_unlock(&ping_pong->mtx);}
pthread_mutex_lock(&ping_pong->mtx);
while (ping_pong->value != PING) { pthread_cond_wait(&ping_pong->cv, &ping_pong->mtx); }
pthread_mutex_unlock(&ping_pong->mtx);
pthread_mutex_destroy(&ping_pong->mtx); pthread_cond_destroy(&ping_pong->cv);
pthread_mutexattr_destroy(&mtxattr); pthread_condattr_destroy(&cvattr);
(void)munmap(ping_pong, sizeof(ping_pong_t)); (void)close(shm_fd);
(void)shm_unlink(SHM_NAME);
return 0;
}
```

```
// pong.cpp: To build: g++ pong.cpp -lpthread -lrt -o pong. Run ping before pong.
#include <fcntl.h>
#include <unistd.h>
#include <iostream>
#include <pthread.h>
#include <sys/mman.h>
#include <semaphore.h>

struct ping_pong_t { int value; pthread_cond_t cv; pthread_mutex_t mtx; };

int main(void) { constexpr int ITERATIONS{10}; constexpr int PING{0}, PONG{1};
const char *const SHM_NAME{"/ping_pong"};
int shm_fd = shm_open(SHM_NAME, O_RDWR, 0666);
if (shm_fd == -1) { std::cout << "Failed to open the shared memory object" <<
std::endl;
std::exit(1); }
if (-1 == ftruncate(shm_fd, sizeof(ping_pong_t))) { std::cout << "Failed to set size of
shared memory object" << std::endl; close(shm_fd); shm_unlink(SHM_NAME);
std::exit(1); }
ping_pong_t *ping_pong = (ping_pong_t *)mmap(nullptr, sizeof(ping_pong_t),
PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0);
if (ping_pong == MAP_FAILED) { std::cout << "Failed to map the shared memory
segment" << std::endl; close(shm_fd); shm_unlink(SHM_NAME); std::exit(1); }
for (int i = 0; i < ITERATIONS; i++) { pthread_mutex_lock(&ping_pong->mtx);
while (ping_pong->value != PONG) { pthread_cond_wait(&ping_pong->cv,
&ping_pong->mtx); }
std::cout << "Pong" << std::endl; ping_pong->value = PING; sleep(1);
pthread_cond_signal(&ping_pong->cv); pthread_mutex_unlock(&ping_pong->mtx); }
(void)munmap(ping_pong, sizeof(ping_pong_t));
(void)close(shm_fd);
return 0;
}
```

Inter-Process Communication

- Some useful links
  - Inter-process communication in Linux
  - What Is Inter-Process Communication In Linux?
  - Inter Process Communication (IPC)
  - Interprocess Communication Mechanisms
  - Overview of POSIX message queues
  - Overview of POSIX shared memory
  - Overview of POSIX semaphores